

https://docs.cloudera.com/documentation/enterprise/5-5-x/topics/cdh_admin_hbase_start_stop.html

sudo hbase master start

sudo hbase regionserver start

Sudo hbase shell

Command	Description
hbase shell	Start the HBase shell
status	Check HBase cluster status
version	Display HBase version
whoami	Show current HBase user
help	List all available commands
help '<command>'	Get help for a specific command
exit or quit	Exit the HBase shell

2. Table Management Commands

Command	Description	Example
create	Create a new table	create 'employee', 'personal', 'professional'
list	List all tables	list
describe	Show table structure	describe 'employee'
disable	Disable a table (required for deletion)	disable 'employee'
enable	Enable a table	enable 'employee'
is_enabled	Check if table is enabled	is_enabled 'employee'
is_disabled	Check if table is disabled	is_disabled 'employee'
alter	Modify table schema	alter 'employee', {NAME => 'salary', VERSIONS => 3}
drop	Delete a table	drop 'employee'
truncate	Delete all table data	truncate 'employee'

3. Data Manipulation

Command	Description
put '<table>', '<row_key>', '<column_family:column>', '<value>'	Insert/update data
get '<table>', '<row_key>'	Retrieve a row
get '<table>', '<row_key>', {COLUMN => '<column_family:column>'}	Get a specific column
scan '<table>'	Scan all rows in a table
scan '<table>', {COLUMNS => ['<column_family:column>']}	Scan specific columns
scan '<table>', {LIMIT => <n>}	Limit scan results
delete '<table>', '<row_key>', '<column_family:column>'	Delete a cell
deleteall '<table>', '<row_key>'	Delete an entire row
count '<table>'	Count rows in a table

Minimum Basic Commands:

Create Table

```
create 'table_name', 'column_family'
```

Store Data

```
put 'table_name' , 'ROW_KEY' , 'column_family:column_name' , 'value'  
put 'table_name' , 'ROW_KEY' , 'column_family:column_name_2' , 'value'
```

Get the value from hbase

```
(select *)      scan  'table_name'  
                get   'table_name','ROW_KEY'
```

Update/Modify

```
put 'table_name' , 'ROW_KEY' , 'column_family:column_name' , 'value_modified'
```

Delete data

```
delete 'table_name' , 'ROW_KEY' , 'column_family:column_name'
```

Drop/alter table

```
Step 1:: disable 'table_name'  
         drop 'table_name'
```

To add Columnfamily:

Alter 'table_name', {NAME=>'new_cf'}

Describe 'tablename'

Creating a Table using HBase Shell

You can create a table using the **create** command, here you must specify the table name and the Column Family name. The **syntax** to create a table in HBase shell is shown below.

```
create '<table name>', '<column family>'
```

Example

Given below is a sample schema of a table named emp. It has two column families: “personal data” and “professional data”.

Row key	personal data	professional data

You can create this table in HBase shell as shown below.

```
hbase(main):002:0> create 'emp', 'personal data', 'professional data'
```

And it will give you the following output.

```
0 row(s) in 1.1300 seconds
=> Hbase::Table - emp
```

Verification

You can verify whether the table is created using the **list** command as shown below. Here you can observe the created emp table.

```
hbase(main):002:0> list
TABLE
emp
2 row(s) in 0.0340 seconds
```

Listing a Table using HBase Shell

list is the command that is used to list all the tables in HBase. Given below is the syntax of the list command.

```
hbase(main):001:0 > list
```

When you type this command and execute in HBase prompt, it will display the list of all the tables in HBase as shown below.

```
hbase(main):001:0> list
TABLE
emp
```

Here you can observe a table named emp.

Disabling a Table using HBase Shell

To delete a table or change its settings, you need to first disable the table using the disable command. You can re-enable it using the enable command.

Given below is the syntax to disable a table:

```
disable 'emp'
```

Example

Given below is an example that shows how to disable a table.

```
hbase(main):025:0> disable 'emp'
0 row(s) in 1.2760 seconds
```

Verification

After disabling the table, you can still sense its existence through **list** and **exists** commands. You cannot scan it. It will give you the following error.

```
hbase(main):028:0> scan 'emp'
ROW          COLUMN + CELL
ERROR: emp is disabled.
```

is_disabled

This command is used to find whether a table is disabled. Its syntax is as follows.

```
hbase> is_disabled 'table name'
```

The following example verifies whether the table named emp is disabled. If it is disabled, it will return true and if not, it will return false.

```
hbase(main):031:0> is_disabled 'emp'
```

```
true
0 row(s) in 0.0440 seconds
```

disable_all

This command is used to disable all the tables matching the given regex. The syntax for **disable_all** command is given below.

```
hbase> disable_all 'r.*'
```

Suppose there are 5 tables in HBase, namely raja, rajani, rajendra, rajesh, and raju. The following code will disable all the tables starting with **raj**.

```
hbase(main):002:07> disable_all 'raj.*'
raja
rajani
rajendra
rajesh
raju
Disable the above 5 tables (y/n)?
y
5 tables successfully disabled
```

Enabling a Table using HBase Shell

Syntax to enable a table:

```
enable 'emp'
```

Example

Given below is an example to enable a table.

```
hbase(main):005:0> enable 'emp'
0 row(s) in 0.4580 seconds
```

Verification

After enabling the table, scan it. If you can see the schema, your table is successfully enabled.

is_enabled

This command is used to find whether a table is enabled. Its syntax is as follows:

```
hbase> is_enabled 'table name'
```

The following code verifies whether the table named **emp** is enabled. If it is enabled, it will return true and if not, it will return false.

```
hbase(main):031:0> is_enabled 'emp'
true
0 row(s) in 0.0440 seconds
```

describe

This command returns the description of the table. Its syntax is as follows:

```
hbase> describe 'table name'
```

Given below is the output of the describe command on the **emp** table.

```
hbase(main):006:0> describe 'emp'
  DESCRIPTION
  ENABLED

'emp', {NAME => 'READONLY', DATA_BLOCK_ENCODING => 'NONE',
BLOOMFILTER
=> 'ROW', REPLICATION_SCOPE => '0', COMPRESSION => 'NONE', VERSIONS
=>
'1', TTL true

=> 'FOREVER', MIN_VERSIONS => '0', KEEP_DELETED_CELLS => 'false',
BLOCKSIZE => '65536', IN_MEMORY => 'false', BLOCKCACHE => 'true'},
{NAME
=> 'personal

data', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW',
REPLICATION_SCOPE => '0', VERSIONS => '5', COMPRESSION => 'NONE',
MIN_VERSIONS => '0', TTL

=> 'FOREVER', KEEP_DELETED_CELLS => 'false', BLOCKSIZE => '65536',
IN_MEMORY => 'false', BLOCKCACHE => 'true'}, {NAME => 'professional
data', DATA_BLO

CK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REPLICATION_SCOPE =>
'0',
VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '0', TTL =>
'FOREVER', K

EEP_DELETED_CELLS => 'false', BLOCKSIZE => '65536', IN_MEMORY =>
'false', BLOCKCACHE => 'true'}, {NAME => 'table_att_unset',
DATA_BLOCK_ENCODING => 'NO

NE', BLOOMFILTER => 'ROW', REPLICATION_SCOPE => '0', COMPRESSION =>
```

```
'NONE', VERSIONS ⇒ '1', TTL ⇒ 'FOREVER', MIN_VERSIONS ⇒ '0',  
KEEP_DELETED_CELLS
```

```
⇒ 'false', BLOCKSIZE ⇒ '6'
```

alter

Alter is the command used to make changes to an existing table. Using this command, you can change the maximum number of cells of a column family, set and delete table scope operators, and delete a column family from a table.

Changing the Maximum Number of Cells of a Column Family

Given below is the syntax to change the maximum number of cells of a column family.

```
hbase> alter 't1', NAME ⇒ 'f1', VERSIONS ⇒ 5
```

In the following example, the maximum number of cells is set to 5.

```
hbase(main):003:0> alter 'emp', NAME ⇒ 'personal data', VERSIONS  
⇒ 5
```

```
Updating all regions with the new schema...
```

```
0/1 regions updated.
```

```
1/1 regions updated.
```

```
Done.
```

```
0 row(s) in 2.3050 seconds
```

Existence of Table using HBase Shell

You can verify the existence of a table using the **exists** command. The following example shows how to use this command.

```
hbase(main):024:0> exists 'emp'  
Table emp does exist
```

```
0 row(s) in 0.0750 seconds
```

```
=====
```

```
hbase(main):015:0> exists 'student'  
Table student does not exist
```

```
0 row(s) in 0.0480 seconds
```

Dropping a Table using HBase Shell

Using the **drop** command, you can delete a table. Before dropping a table, you have to disable it.

```
hbase(main):018:0> disable 'emp'  
0 row(s) in 1.4580 seconds
```

```
hbase(main):019:0> drop 'emp'  
0 row(s) in 0.3060 seconds
```

Verify whether the table is deleted using the exists command.

```
hbase(main):020:0>gt; exists 'emp'  
Table emp does not exist  
0 row(s) in 0.0730 seconds
```

drop_all

This command is used to drop the tables matching the “regex” given in the command. Its syntax is as follows:

```
hbase> drop_all 't.*'
```

Note: Before dropping a table, you must disable it.

Example

Assume there are tables named raja, rajani, rajendra, rajesh, and raju.

```
hbase(main):017:0> list
```

```
TABLE
raja
rajani
rajendra
rajesh
raju
9 row(s) in 0.0270 seconds
```

All these tables start with the letters **raj**. First of all, let us disable all these tables using the **disable_all** command as shown below.

```
hbase(main):002:0> disable_all 'raj.*'
raja
rajani
rajendra
rajesh
raju
Disable the above 5 tables (y/n)?
Y
5 tables successfully disabled
```

Now you can delete all of them using the **drop_all** command as given below.

```
hbase(main):018:0> drop_all 'raj.*'
raja
rajani
rajendra
rajesh
raju
Drop the above 5 tables (y/n)?
Y
5 tables successfully dropped
```

exit

You exit the shell by typing the **exit** command.

```
hbase(main):021:0> exit
```

Stopping HBase

To stop HBase, browse to the HBase home folder and type the following command.

```
./bin/stop-hbase.sh
```

Inserting Data using HBase Shell

Using **put** command, you can insert rows into a table. Its syntax is as follows:

```
put '<table name>', 'row1', '<colfamily:colname>', '<value>'
```

Inserting the First Row

Let us insert the first row values into the emp table as shown below.

```
hbase(main):005:0> put 'emp','1','personal data:name','raju'
0 row(s) in 0.6600 seconds
hbase(main):006:0> put 'emp','1','personal data:city','hyderabad'
0 row(s) in 0.0410 seconds
hbase(main):007:0> put 'emp','1','professional
data:designation','manager'
0 row(s) in 0.0240 seconds
hbase(main):007:0> put 'emp','1','professional
data:salary','50000'
0 row(s) in 0.0240 seconds
```

Insert the remaining rows using the put command in the same way. If you insert the whole table, you will get the following output.

```
hbase(main):022:0> scan 'emp'

      ROW                                         COLUMN+CELL
1 column=personal data:city, timestamp=1417524216501,
value=hyderabad

1 column=personal data:name, timestamp=1417524185058, value=ramu

1 column=professional data:designation, timestamp=1417524232601,
value=manager

1 column=professional data:salary, timestamp=1417524244109,
value=50000

2 column=personal data:city, timestamp=1417524574905,
value=chennai

2 column=personal data:name, timestamp=1417524556125, value=ravi

2 column=professional data:designation, timestamp=1417524592204,
value=sr:engg

2 column=professional data:salary, timestamp=1417524604221,
value=30000

3 column=personal data:city, timestamp=1417524681780, value=delhi
```

```

3 column=personal data:name, timestamp=1417524672067,
value=rajesh

3 column=professional data:designation, timestamp=1417524693187,
value=jr:engg
3 column=professional data:salary, timestamp=1417524702514,
value=25000

```

Updating Data using HBase Shell

You can update an existing cell value using the **put** command. To do so, just follow the same syntax and mention your new value as shown below.

```
put 'table name','row ','Column family:column name','new value'
```

The newly given value replaces the existing value, updating the row.

Example

Suppose there is a table in HBase called **emp** with the following data.

```

hbase(main):003:0> scan 'emp'
ROW          COLUMN + CELL
row1 column = personal:name, timestamp = 1418051555, value = raju
row1 column = personal:city, timestamp = 1418275907, value =
Hyderabad
row1 column = professional:designation, timestamp =
14180555,value = manager
row1 column = professional:salary, timestamp =
1418035791555,value = 50000
1 row(s) in 0.0100 seconds

```

The following command will update the city value of the employee named 'Raju' to Delhi.

```

hbase(main):002:0> put 'emp','row1','personal:city','Delhi'
0 row(s) in 0.0400 seconds

```

The updated table looks as follows where you can observe the city of Raju has been changed to 'Delhi'.

```

hbase(main):003:0> scan 'emp'
ROW          COLUMN + CELL
row1 column = personal:name, timestamp = 1418035791555, value =
raju
row1 column = personal:city, timestamp = 1418274645907, value =
Delhi

```

```

row1 column = professional:designation, timestamp =
141857555,value = manager
row1 column = professional:salary, timestamp = 1418039555, value
= 50000
1 row(s) in 0.0100 seconds

```

Reading Data using HBase Shell

The **get** command and the **get()** method of **HTable** class are used to read data from a table in HBase. Using **get** command, you can get a single row of data at a time. Its syntax is as follows:

```
get '<table name>','row1'
```

Example

The following example shows how to use the get command. Let us scan the first row of the **emp** table.

```

hbase(main):012:0> get 'emp', '1'

      COLUMN                                CELL

personal : city timestamp = 1417521848375, value = hyderabad
personal : name timestamp = 1417521785385, value = ramu
professional: designation timestamp = 1417521885277, value =
manager
professional: salary timestamp = 1417521903862, value = 50000

4 row(s) in 0.0270 seconds

```

Reading a Specific Column

Given below is the syntax to read a specific column using the **get** method.

```
hbase> get 'table name', 'rowid', {COLUMN => 'column
family:column name '}
```

Example

Given below is the example to read a specific column in HBase table.

```

hbase(main):015:0> get 'emp', 'row1', {COLUMN => 'personal:name'}
      COLUMN                                CELL
personal:name timestamp = 1418035791555, value = raju
1 row(s) in 0.0080 seconds

```

Deleting a Specific Cell in a Table

Using the **delete** command, you can delete a specific cell in a table. The syntax of **delete** command is as follows:

```
delete '<table name>', '<row>', '<column name >', '<time stamp>'
```

Example

Here is an example to delete a specific cell. Here we are deleting the salary.

```
hbase(main):006:0> delete 'emp', '1', 'personal data:city',  
1417521848375  
0 row(s) in 0.0060 seconds
```

Deleting All Cells in a Table

Using the “deleteall” command, you can delete all the cells in a row. Given below is the syntax of deleteall command.

```
deleteall '<table name>', '<row>',
```

Example

Here is an example of “deleteall” command, where we are deleting all the cells of row1 of emp table.

```
hbase(main):007:0> deleteall 'emp','1'  
0 row(s) in 0.0240 seconds
```

Verify the table using the **scan** command. A snapshot of the table after deleting the table is given below.

```
hbase(main):022:0> scan 'emp'
```

ROW	COLUMN + CELL
-----	---------------

2	column = personal data:city, timestamp = 1417524574905, value = chennai
---	---

2	column = personal data:name, timestamp = 1417524556125, value = ravi
---	--

2	column = professional data:designation, timestamp = 1417524204, value = sr:engg
---	---

```

2 column = professional data:salary, timestamp = 1417524604221,
value = 30000

3 column = personal data:city, timestamp = 1417524681780, value =
delhi

3 column = personal data:name, timestamp = 1417524672067, value =
rajesh

3 column = professional data:designation, timestamp = 1417523187,
value = jr:engg

3 column = professional data:salary, timestamp = 1417524702514,
value = 25000

```

Scanning using HBase Shell

The **scan** command is used to view the data in HTable. Using the scan command, you can get the table data. Its syntax is as follows:

```
scan '<table name>'
```

Example

The following example shows how to read data from a table using the scan command. Here we are reading the **emp** table.

```

hbase(main):010:0> scan 'emp'

ROW                                COLUMN + CELL

1 column = personal data:city, timestamp = 1417521848375, value =
hyderabad

1 column = personal data:name, timestamp = 1417521785385, value =
ramu

1 column = professional data:designation, timestamp =
1417585277,value = manager

1 column = professional data:salary, timestamp = 1417521903862,
value = 50000

1 row(s) in 0.0370 seconds

```

count

You can count the number of rows of a table using the **count** command. Its syntax is as follows:

```
count '<table name>'
```

After deleting the first row, emp table will have two rows. Verify it as shown below.

```
hbase(main):023:0> count 'emp'
2 row(s) in 0.090 seconds
⇒ 2
```

truncate

This command disables drops and recreates a table. The syntax of **truncate** is as follows:

```
hbase> truncate 'table name'
```

Example

Given below is the example of truncate command. Here we have truncated the **emp** table.

```
hbase(main):011:0> truncate 'emp'
Truncating 'one' table (it may take a while):
- Disabling table...
- Truncating table...
0 row(s) in 1.5950 seconds
```

After truncating the table, use the scan command to verify. You will get a table with zero rows.

```
hbase(main):017:0> scan 'emp'
ROW          COLUMN + CELL
0 row(s) in 0.3110 seconds
```

Versions:

get 'table_name', 'ROW_KEY',{COLUMN=>'column_family:column',VERSIONS=>'no of versions to be displayed'}

Example:

1 row(s) in 0.0700 seconds

```
hbase(main):014:0> get 'htest' , 'r1' , {COLUMN=>'cf:c3',VERSIONS=>3}
COLUMN                                CELL
cf:c3                                timestamp=1414254592531, value=v3_updated_3
cf:c3                                timestamp=1414254579862, value=v3_updated
cf:c3                                timestamp=1414254290246, value=v3
3 row(s) in 0.0460 seconds
```

```
hbase(main):015:0> get 'htest' , 'r1' , {COLUMN=>'cf:c3',VERSIONS=>2}
COLUMN                                CELL
cf:c3                                timestamp=1414254592531, value=v3_updated_3
cf:c3                                timestamp=1414254579862, value=v3_updated
```